

0.67.3. Les variables de la lettre dans leur propre fichier — en simple

Le 0.67 en simple : `variables.h` ne garde que `x` et `y` ; `principal.cpp`, une ligne dans la boucle.

1. Ce qu'il faut comprendre

C'est le 0.67, en plus simple : la boucle de la croix est remplacée par une ligne, `deplace_croix`.

`variables.h` rétrécit aussi : plus besoin d'`attente`, la console la tient elle-même. Il ne reste que `x` et `y`, la place de la lettre.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1748 octets** de cartouche.

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

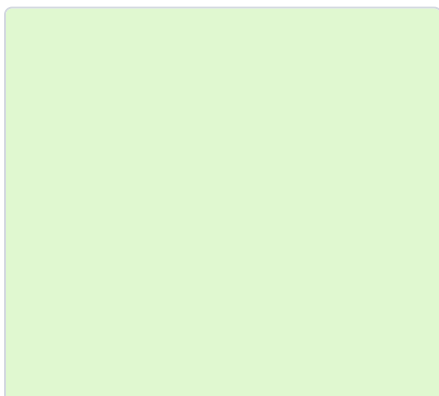
```
// x et y : voir l'onglet variables.h
#include "variables.h"
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

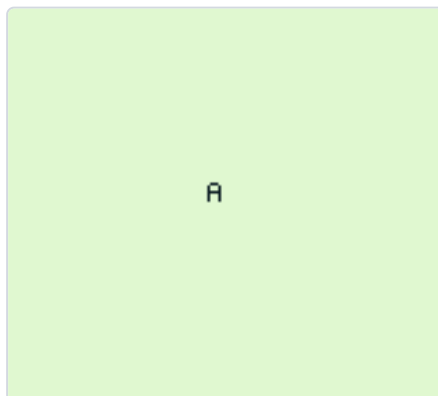
```
int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 133);
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Comme au 0.67 : un A qui se déplace avec les flèches, programme en deux onglets.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ la croix le déplace

✓ il est à sa place

Ensuite : **0.68. Voir la position du A en direct** — niveau 0 sur 10. · [gameboy3](#) — écrire une cartouche Game Boy en C++